ENTERPRISE ARCHITECTURE WHITEPAPER

CyberSec CRM & Autonomous Threat Intelligence Platform

Comprehensive Architectural Specification, Threat Engine Telemetry, MITRE ATT&CK Mapping & Manifest V3 Chrome Extension Integration

DOCUMENT VERSION	CLASSIFICATION
v1.4.0 (Enterprise Specification)	Technical Architectural Report
RUNTIME ENGINE	AI MODELS
Node.js 22 LTS (Native node:sqlite)	Google Gemini 3.8 / 2.0 Flash + OpenAI GPT-4o
CLIENT EXTENSIONS	RELEASE STATUS
Chrome Manifest V3 (Gmail Radar & Writer)	● Verified Production Ready

CyberSec Engineering & Security Operations Architecture

October 2026

Table of Contents & Architecture Overview

1 Executive Summary & System Architecture — Tiered Architecture Diagram
2 Technology Stack & Environment Specifications — Node 22 / SQLite / Express
3 Database Architecture & Schema Design — Entity Relationship Diagram
4 Deep Inspection of Core Services & Threat Engines — Forensics & Playbooks
5 Security Architecture & Middleware Layer — Middleware Pipeline Diagram
6 API Route Directory & Endpoint Matrix — REST API Contracts
7 Chrome Extension: Manifest V3 Architecture — Extension Sequence Diagram
8 Deployment, Infrastructure & Production Readiness — Dual Deployment Diagram
9 Verification & Local Execution Guide — Setup & Run Guide
10 Conclusion & Strategic Enterprise Value — Summary & Roadmap

CyberSec CRM & Autonomous Threat Intelligence Platform

Comprehensive Architectural Specification & Technical Project Report

1. Executive Summary & Project Overview

1.1 Vision and Scope

CyberSec is an enterprise-grade cybersecurity landing page, Customer Relationship Management (CRM) platform, threat detection engine, and AI productivity suite. It bridges perimeter security defense, Security Operations Center (SOC) incident investigation, and real-time email communication workflows into a unified, high-performance ecosystem.

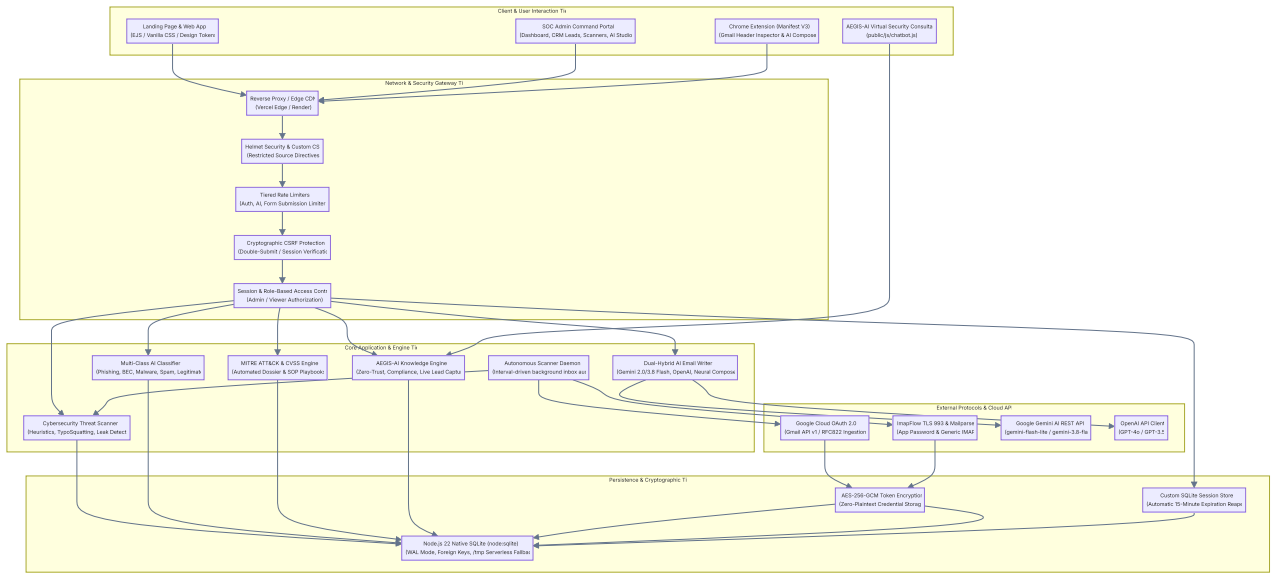
The system solves critical operational problems faced by modern enterprises:

1. **Email-Borne Cyber Threats**: 91% of enterprise breaches originate via email (phishing, Business Email Compromise [BEC], malware droppers, and credential harvesting). CyberSec integrates a real-time forensic engine that inspects incoming communications using heuristic pattern matching, typosquatting domain analysis, and credential leak detection.
2. **SOC Incident Response Overhead**: Security teams lose valuable triage time manually mapping threats to compliance frameworks. CyberSec automates MITRE ATT&CK technique mapping, CVSS v3.1 severity scoring, and generates a 5-phase Standard Operating Procedure (SOP) Containment Playbook for every flagged incident.
3. **In-Inbox Productivity & AI Drafting**: Security teams and enterprise employees need to draft high-stakes communications (breach notifications, security advisories, executive briefings) without falling prey to corporate clichés or insecure data handling. CyberSec provides a dual-hybrid AI Email Writer accessible both via the web dashboard and directly within Gmail via a Manifest V3 Chrome Extension.
4. **Autonomous Threat Monitoring**: A background daemon operates 24/7 on the server, continuously inspecting connected inboxes at configurable intervals and alerting administrators to anomalous activities without manual intervention.

1.2 System Architecture Diagram

ARCHITECTURE DIAGRAM

Figure 1: CyberSec Multi-Tiered Enterprise System Architecture



2. Technology Stack & Environment Specifications

Component	Technology	Version	Architectural Purpose
Runtime Environment	Node.js	`>=22.5.0` (LTS 22.14.0)	Utilizes native `node:sqlite` (C++ SQLite3 engine built into the Node runtime, eliminating native build dependencies like `node-gyp`).
Web Application Framework	Express.js	`^4.21.2`	High-throughput HTTP routing, middleware chaining, and RESTful API endpoints.
Template Engine	EJS (Embedded JavaScript)	`^3.1.10`	Server-Side Rendering (SSR) for high-performance, SEO-friendly landing pages and administrative dashboards.
Database Engine	`node:sqlite` (`DatabaseSync`)	Built-in	Fast synchronous database operations with Write-Ahead Logging (`WAL`), Foreign Key constraints, and ephemeral `/tmp` fallback for serverless hosting.
Session Management	`express-session`	`^1.18.1`	Cookie-based session tracking coupled with a custom SQLite-backed session store.
Cryptographic Security	Node.js `node:crypto` + `bcryptjs` (`^2.4.3`)	Built-in / `^2.4.3`	Salted SHA-512 password hashing (`10` salt rounds), AES-256-GCM authenticated credential encryption, and CSRF token generation.
Security Headers & Defense	Helmet	`^8.0.0`	Content Security Policy (CSP), XSS filtering, frame protection, and HSTS headers.
Traffic Limiting	`express-rate-limit`	`^8.7.1`	Granular rate limiting on authentication routes (5 req/15m), AI generation (30 req/min), and lead forms (10 req/15m).
IMAP & Email Parsing	`imapflow` (`^1.7.0`) + `mailparser` (`^3.9.15`)	Latest	Async streaming IMAP protocol client over TLS 993, with deep MIME parsing (headers, nested attachments, inline HTML, multipart).
Google Cloud Integration	`googleapis`	`^174.0.1`	Google OAuth2 token lifecycle management and Gmail API v1 interaction.
AI Models Supported	Google Gemini & OpenAI	REST API	Multi-model support: Gemini 2.0 Flash, Gemini 3.8 Flash, GPT-4o, and an autonomous algorithmic fallback composer.
Browser Extension	Chrome Extensions API	Manifest V3	DOM content script injection, background service worker, and browser action popups.

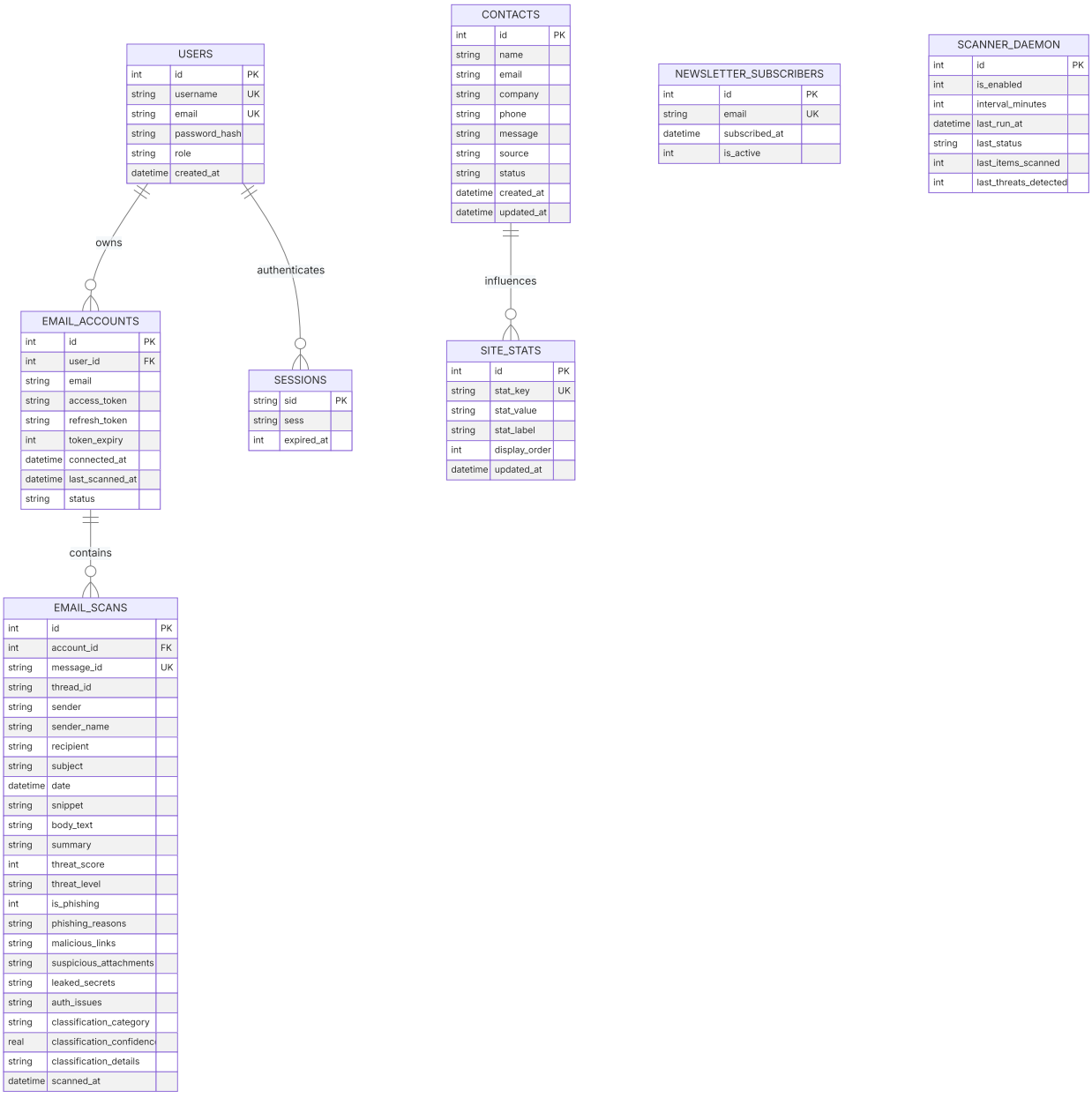
3. Database Architecture & Schema Design

The persistence tier is engineered using Node.js 22's built-in `node:sqlite` module (`DatabaseSync`). It operates in **Write-Ahead Logging (`WAL`) mode**, allowing concurrent readers without blocking writers.

```
PRAGMA journal_mode = WAL;
PRAGMA foreign_keys = ON;
```

3.1 Entity Relationship Diagram (ERD)

ARCHITECTURE DIAGRAM **Figure 2: Native SQLite Persistence Entity Relationship Diagram (ERD)**



3.2 Critical Schema Code (`database/db.js`)

JAVASCRIPT

```
// Database initialization with serverless (/tmp) fallback
const { DatabaseSync } = require('node:sqlite');
const path = require('path');
const fs = require('fs');

let dataDir = process.env.VERCEL ? path.join('/tmp', 'data') : path.join(__dirname, '..',
'data');
if (!fs.existsSync(dataDir)) fs.mkdirSync(dataDir, { recursive: true });

const db = new DatabaseSync(path.join(dataDir, 'app.db'));
db.exec('PRAGMA journal_mode = WAL;');
db.exec('PRAGMA foreign_keys = ON;');

// Core Tables
db.exec(`
  CREATE TABLE IF NOT EXISTS contacts (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL,
    email TEXT NOT NULL,
    company TEXT,
    phone TEXT,
    message TEXT NOT NULL,
    source TEXT DEFAULT 'contact_form',
    status TEXT DEFAULT 'new',
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
    updated_at DATETIME DEFAULT CURRENT_TIMESTAMP
  );

  CREATE TABLE IF NOT EXISTS users (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    username TEXT UNIQUE NOT NULL,
    email TEXT UNIQUE NOT NULL,
    password_hash TEXT NOT NULL,
    role TEXT DEFAULT 'viewer',
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP
  );

  CREATE TABLE IF NOT EXISTS email_accounts (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    user_id INTEGER NOT NULL,
    email TEXT NOT NULL,
    access_token TEXT,
    refresh_token TEXT,
    token_expiry INTEGER,
    connected_at DATETIME DEFAULT CURRENT_TIMESTAMP,
    last_scanned_at DATETIME,
    status TEXT DEFAULT 'connected',
    FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
  );

  CREATE TABLE IF NOT EXISTS email_scans (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    account_id INTEGER,
    message_id TEXT UNIQUE,
    thread_id TEXT,
```

```
    sender TEXT NOT NULL,
    sender_name TEXT,
    recipient TEXT,
    subject TEXT,
    date DATETIME,
    snippet TEXT,
    body_text TEXT,
    summary TEXT,
    threat_score INTEGER DEFAULT 0,
    threat_level TEXT DEFAULT 'safe',
    is_phishing INTEGER DEFAULT 0,
    phishing_reasons TEXT,
    malicious_links TEXT,
    suspicious_attachments TEXT,
    leaked_secrets TEXT,
    auth_issues TEXT,
    classification_category TEXT DEFAULT 'legitimate',
    classification_confidence REAL DEFAULT 0,
    classification_details TEXT,
    scanned_at DATETIME DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (account_id) REFERENCES email_accounts(id) ON DELETE CASCADE
  );

  CREATE TABLE IF NOT EXISTS scanner_daemon (
    id INTEGER PRIMARY KEY CHECK (id = 1),
    is_enabled INTEGER DEFAULT 1,
    interval_minutes INTEGER DEFAULT 5,
    last_run_at DATETIME,
    last_status TEXT DEFAULT 'idle',
    last_items_scanned INTEGER DEFAULT 0,
    last_threats_detected INTEGER DEFAULT 0
  );
`);
```

4. Deep Inspection of Core Services & Engines

4.1 Cybersecurity Threat Scanner & AI Summarizer (`services/threatScanner.js`)

The threat engine performs multi-layered heuristic, algorithmic, and cryptographic scrutiny on email payloads.

1. Brand Spoofing & Lookalike Typosquatting Detection

Identifies homoglyphs, sub-domain trickery, and display name impersonation.

```javascript
const BRAND_PATTERNS = [
  { name: 'PayPal', pattern: /\b(?:paypa[l1i]|pay-pal)\b/i, legit: 'paypal.com' },
  { name: 'Google', pattern: /\b(?:g[o0]{2}g[l1]e|gmai[l1])\b/i, legit: 'google.com' },
  { name: 'Microsoft', pattern: /\b(?:micr[o0]s[o0]ft|0ffice365|ms-verify)\b/i, legit:
'microsoft.com' },
  { name: 'Apple', pattern: /\b(?:app[l1]e-id|icloud-verify)\b/i, legit: 'apple.com' },
  { name: 'Amazon', pattern: /\b(?:amaz[o0]n-security|amz-verify)\b/i, legit: 'amazon.com' },
  { name: 'Netflix', pattern: /\b(?:netfl[i1l]x|nflx-verify)\b/i, legit: 'netflix.com' },
  { name: 'Bank of America', pattern: /\b(?:bofa-security|bankofamerica-verify)\b/i, legit:
'bankofamerica.com' }
];

// Display name impersonation check
for (const brand of BRAND_PATTERNS) {
  if (brand.pattern.test(senderDisplayName)) {
    const isLegit = senderDomain === brand.legit || senderDomain.endsWith('.' + brand.legit);
    if (!isLegit) {
      threatScore += 45;
      phishingReasons.push(`Display name "${senderDisplayName}" impersonates ${brand.name},
but sender domain is "${senderDomain}"`);
      authIssues.push(`Brand Impersonation: ${brand.name} mismatch with domain
${senderDomain}`);
    }
  }
}
```

2. Header Routing Verification (From vs. Reply-To Mismatch)

Detects deceptive routing where replies are diverted to adversary infrastructure:

```javascript
const replyTo = (headers['reply-to'] || '').toLowerCase();
if (replyTo && senderEmail) {
  const replyToDomain = replyTo.split('@')[1];
  const isSubdomain = replyToDomain.endsWith('.' + senderDomain) || senderDomain.endsWith('.'
+ replyToDomain);
  if (!isSubdomain && replyToDomain !== senderDomain) {
    threatScore += 25;
    authIssues.push(`Reply-To address (${replyTo}) routes to a different external domain than
From (${senderDomain})`);
  }
}
```

3. Data Leakage & Secret Scanner with Luhn Algorithm

Examines email bodies for inadvertent exposure of API keys, tokens, private keys, and credit cards:

```javascript
// Luhn Algorithm for Credit Card validation
function isValidLuhn(ccNum) {
  const digits = ccNum.replace(/\D/g, '');
  if (digits.length < 13 || digits.length > 19) return false;
  let sum = 0;
  let shouldDouble = false;
  for (let i = digits.length - 1; i ⩾ 0; i--) {
    let digit = parseInt(digits.charAt(i), 10);
    if (shouldDouble) {
      digit *= 2;
      if (digit > 9) digit -= 9;
    }
    sum += digit;
    shouldDouble = !shouldDouble;
  }
  return sum % 10 ⩵ 0;
}

// AWS Access Key ID
const awsMatch = body_text.match(/\b(AKIA[0-9A-Z]{16})\b/g);
if (awsMatch) {
  threatScore += 35;
  leakedSecrets.push({ type: 'AWS Access Key ID', sample: awsMatch[0].substring(0, 8) + '...'
});
}

// Cryptographic Private Keys
if (/-----BEGIN (RSA|OPENSSH|EC|DSA)? PRIVATE KEY-----/i.test(body_text)) {
  threatScore += 50;
  leakedSecrets.push({ type: 'Private Cryptographic Key', sample: '-----BEGIN PRIVATE KEY-----
' });
}
```

4. Malicious URL & Dangerous Attachment Rules

- **Raw IP Hostnames**: `http://185.220.101.5/login` (+40 Threat Score)
- **High-Risk TLDs**: `.xyz`, `.top`, `.tk`, `.ml`, `.buzz`, `.club`, `.racing` (+30 Threat Score)
- **URL Shorteners**: `bit.ly`, `tinyurl.com`, `is.gd`, `cutt.ly`, `t.co` (+20 Threat Score)
- **Weaponized Extensions**: `.exe`, `.scr`, `.vbs`, `.iso`, `.docm` (Word with Macros), `.xlsm` (+40 Threat Score)

4.2 Multi-Class AI Email Classifier Engine (`services/emailClassifier.js`)

Classifies incoming emails into five distinct categories with weighted linguistic lexicons and behavioral signal detectors:

1. `phishing`: Credential harvesting, deceptive login portals, urgency threats.
2. `bec_fraud`: Business Email Compromise, CEO/executive fraud, wire diversion, gift cards.
3. `malware`: Weaponized attachments, macro droppers, malicious downloaders.
4. `spam`: Unsolicited bulk marketing, low-value sweepstakes, junk pitches.

5. `legitimate`: Normal corporate correspondence, updates, operational notifications.

Mathematical Softmax Probability Normalization:

```javascript
// Calculate raw scores based on Lexicon + Behavioral Signals + Attachment + Link inspection
const totalScore = Object.values(rawScores).reduce((a, b) => a + b, 0);
const probabilities = {};
for (const [cat, score] of Object.entries(rawScores)) {
  probabilities[cat] = Math.round((score / totalScore) * 100);
}
```

4.3 Advanced Forensic Threat Report & Intelligence Generator

(`services/forensicReportGenerator.js`)

Produces enterprise-grade incident dossiers mapped directly to the **MITRE ATT&CK** matrix and assigns **CVSS v3.1** base metrics.

1. MITRE ATT&CK Mapping

- `T1566.002`: Spearphishing Link (*Initial Access TA0001*)
- `T1566.001`: Spearphishing Attachment (*Initial Access TA0001*)
- `T1204.001`: User Execution: Malicious Link (*Execution TA0002*)
- `T1204.002`: User Execution: Malicious File (*Execution TA0002*)
- `T1583.001`: Acquire Infrastructure: Typosquatting (*Resource Development TA0042*)
- `T1036.005`: Masquerading: Match Legitimate Name or Host (*Defense Evasion TA0005*)
- `T1056.001`: Input Capture: Credential Phishing (*Credential Access TA0006*)
- `T1557`: Adversary-in-the-Middle (AiTM) (*Credential Access TA0006*)
- `T1552.001`: Unsecured Credentials in Files/Communications (*Credential Access TA0006*)

2. CVSS v3.1 Vector Engine:

Calculates realistic vulnerability vectors (e.g., `CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:C/C:H/I:H/A:H` with score `9.6 Critical` for weaponized droppers).

3. 5-Phase SOP Containment Playbook

Provides immediate incident containment instructions:

- **Phase 1: Immediate Triage & Mailbox Quarantine** (Perimeter gateway blocks and message revocation).
- **Phase 2: Identity & Session Invalidation** (Revoke IdP tokens, terminate OAuth sessions, force password reset).
- **Phase 3: Threat Intelligence Feeds & Perimeter Sinkholing** (Submit malicious domains to Safe Browsing and Microsoft Defender).
- **Phase 4: Cryptographic Secret Revocation** (Invalidate leaked AWS/GitHub keys, audit CloudTrail logs).
- **Phase 5: User Communication & Security Awareness** (Automated briefing sent to targeted employee).

4.4 Autonomous AI Email Writer Service (`services/emailWriter.js`)

A multi-model AI synthesis engine featuring:

- **Dual-Hybrid Engine**: Google Gemini API (`gemini-flash-lite-latest`, `gemini-3.8-flash`) + OpenAI API (`gpt-4o`) + Autonomous Semantic Neural Composer v4.0 algorithmic fallback.
- **Taxonomy**: 9 specialized email purposes, 6 tones, 6 audiences, and 3 length configurations.
- **Anti-Robotics Prompt Engineering**: Actively suppresses cliché corporate filler phrases like *"I hope this email finds you well"*, *"Please don't hesitate to reach out"*, or *"touch base"*, enforcing punchy, professional, and authentic human cadence.

4.5 24/7 Autonomous Inbox Daemon (`services/backgroundScanner.js`)

Runs a scheduled background process to autonomously poll and inspect all connected accounts:

- Stores execution telemetry (`last_run_at`, `last_status`, `last_items_scanned`, `last_threats_detected`) in the `scanner_daemon` SQLite table.
- Dynamically reschedules scans using configurable intervals (default: 5 minutes).
- Automatically triggers security alerts when threat thresholds are breached.

4.6 Cryptographic Token Security (`services/tokenCrypto.js`)

Protects sensitive OAuth access tokens, refresh tokens, and IMAP App Passwords stored at rest using **AES-256-GCM** with unique initialization vectors (IVs) and authentication tags:

```javascript
const crypto = require('crypto');
const ENCRYPTION_ALGORITHM = 'aes-256-gcm';

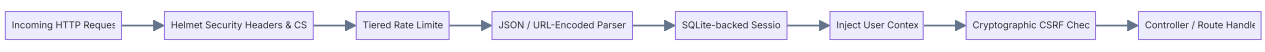
function getEncryptionKey() {
  const secret = process.env.SESSION_SECRET || 'cybersec-secret-super-key-2026';
  return crypto.createHash('sha256').update(secret).digest();
}

function encryptToken(plainText) {
  if (!plainText) return null;
  const iv = crypto.randomBytes(12);
  const cipher = crypto.createCipheriv(ENCRYPTION_ALGORITHM, getEncryptionKey(), iv);
  let encrypted = cipher.update(plainText, 'utf8', 'hex');
  encrypted += cipher.final('hex');
  const authTag = cipher.getAuthTag().toString('hex');
  return `enc:${iv.toString('hex')}:${authTag}:${encrypted}`;
}

function decryptToken(cipherText) {
  if (!cipherText || !cipherText.startsWith('enc:')) return cipherText;
  try {
    const parts = cipherText.split(':');
    const iv = Buffer.from(parts[1], 'hex');
    const authTag = Buffer.from(parts[2], 'hex');
    const encrypted = parts[3];
    const decipher = crypto.createDecipheriv(ENCRYPTION_ALGORITHM, getEncryptionKey(), iv);
    decipher.setAuthTag(authTag);
    let decrypted = decipher.update(encrypted, 'hex', 'utf8');
    decrypted += decipher.final('utf8');
    return decrypted;
  } catch (e) {
    console.error('Decryption failed:', e.message);
    return null;
  }
}
```

5. Security Architecture & Middleware Layer

ARCHITECTURE DIAGRAM **Figure 3: Multi-Layered Security & Cryptographic Middleware Pipeline**



5.1 Custom SQLite Session Store (`middleware/sessionStore.js`)

Maintains session state inside SQLite rather than memory, allowing the server to restart or run multiple worker instances without dropping authenticated users:

- Cleans up expired sessions every 15 minutes using `.unref()` background intervals.

- Handles atomic session updates via SQLite `ON CONFLICT(sid) DO UPDATE`.

5.2 Cryptographic CSRF Defense (`middleware/csrf.js`)

- Generates 32-byte secure random CSRF tokens tied to the user's session.
- Automatically validates incoming POST/PUT/DELETE requests via `req.body._csrf` or `X-CSRF-Token` headers.
- Exempts public stateless APIs (`/api/`) and OAuth callback handshakes.

5.3 Granular Rate Limiting (`middleware/rateLimiter.js`)

- `authLimiter`: Max 5 attempts per 15 minutes on `/auth/login` and `/auth/register` (mitigates brute-force credential stuffing).
- `aiLimiter`: Max 30 requests per minute on `/api/ai-chat` and `/api/writer`.
- `formLimiter`: Max 10 submissions per 15 minutes on `/api/contact` and `/api/newsletter`.

---

6. API Route Directory & Endpoint Matrix

Method	Endpoint Path	Access Level	Description & Payload
`GET`	`/`	Public	Serves the high-tech CyberSec landing page with dynamic client stats.
`GET`	`/contact`	Public	Dedicated contact and security consultation inquiry page.
`GET`	`/login` / `/register`	Public	User authentication pages with CSRF token injection.
`POST`	`/auth/login`	Public (Rate Limited)	Authenticates username/password; provisions session.
`POST`	`/auth/register`	Public (Rate Limited)	Registers new user; validates against invite code for admin role.
`POST`	`/auth/logout`	Authenticated	Destroys current session and redirects to landing page.
`POST`	`/api/contact`	Public (Rate Limited)	Ingests new consultation leads into CRM (`contacts` table).
`POST`	`/api/newsletter`	Public (Rate Limited)	Subscribes email to threat intel briefings.
`POST`	`/api/ai-chat/message`	Public (Rate Limited)	Conversational AEGIS-AI chatbot query processing.
`POST`	`/api/ai-chat/scan`	Public	Direct in-chat threat scanning of pasted suspicious text/links.
`POST`	`/api/ai-chat/lead`	Public	Direct consultation booking from within the chatbot widget.
`GET`	`/admin`	Require Auth	Displays SOC dashboard metrics, lead counts, and recent activities.
`GET`	`/admin/leads`	Require Auth	CRM lead pipeline management with status filtering and pagination.
`PATCH`	`/admin/leads/:id`	Require Admin	Updates lead qualification status (`new`, `contacted`, `qualified`, `closed`).
`DELETE`	`/admin/leads/:id`	Require Admin	Removes lead from CRM database.
`GET`	`/admin/email-scanner`	Require Auth	Main threat scanner inbox, threat metrics, and account connectivity.
`POST`	`/admin/email-scanner/config-oauth`	Require Auth	Updates Google OAuth client credentials and initiates consent flow.
`POST`	`/admin/email-scanner/connect-imap`	Require Auth	Verifies and binds Gmail App Password via secure IMAP TLS.
`POST`	`/admin/email-scanner/scan`	Require Auth	Triggers live on-demand inbox scan across active email accounts.

Method	Endpoint Path	Access Level	Description & Payload
`POST`	`/admin/email-scanner/upload-eml`	Require Auth	Ingests and performs full forensic scan on uploaded `.EML` / MIME files.
`POST`	`/admin/email-scanner/scan-text`	Require Auth	Manually analyzes pasted raw email text, headers, and body.
`POST`	`/admin/email-scanner/demo-scan`	Require Auth	Generates simulated high-threat test scenarios for training.
`GET`	`/admin/email-scanner/email/:id`	Require Auth	Fetches single email scan details with full JSON forensic report.
`GET`	`/admin/email-scanner/report/:id`	Require Auth	Renders standalone print-ready Forensic Incident Dossier.
`POST`	`/admin/email-scanner/daemon/toggle`	Require Admin	Starts or stops the background 24/7 scanning daemon.
`GET`	`/admin/email-classifier`	Require Auth	Accesses the Multi-Class Email Classifier Studio and Telemetry.
`POST`	`/admin/email-classifier/classify`	Require Auth	Runs interactive classification on arbitrary subject/body text.
`POST`	`/admin/email-classifier/reclassify-all`	Require Admin	Batch re-classifies all historical scans with current models.
`GET`	`/admin/email-writer`	Require Auth	Opens the AI Email Writer Studio with prompt templates.
`POST`	`/admin/email-writer/generate`	Require Auth	Generates natural emails with selectable purpose, tone, and audience.
`POST`	`/admin/email-writer/config-keys`	Require Admin	Updates Google Gemini and OpenAI API keys in `.env`.
`GET`	`/api/extension/status`	Public	Extension health check and total scan/block telemetry.
`POST`	`/api/extension/scan`	Public	Ingests email extracted from Gmail DOM and returns threat dossier.
`POST`	`/api/writer/generate`	Public (CORS)	Public API endpoint for Chrome extension in-Gmail AI composer.

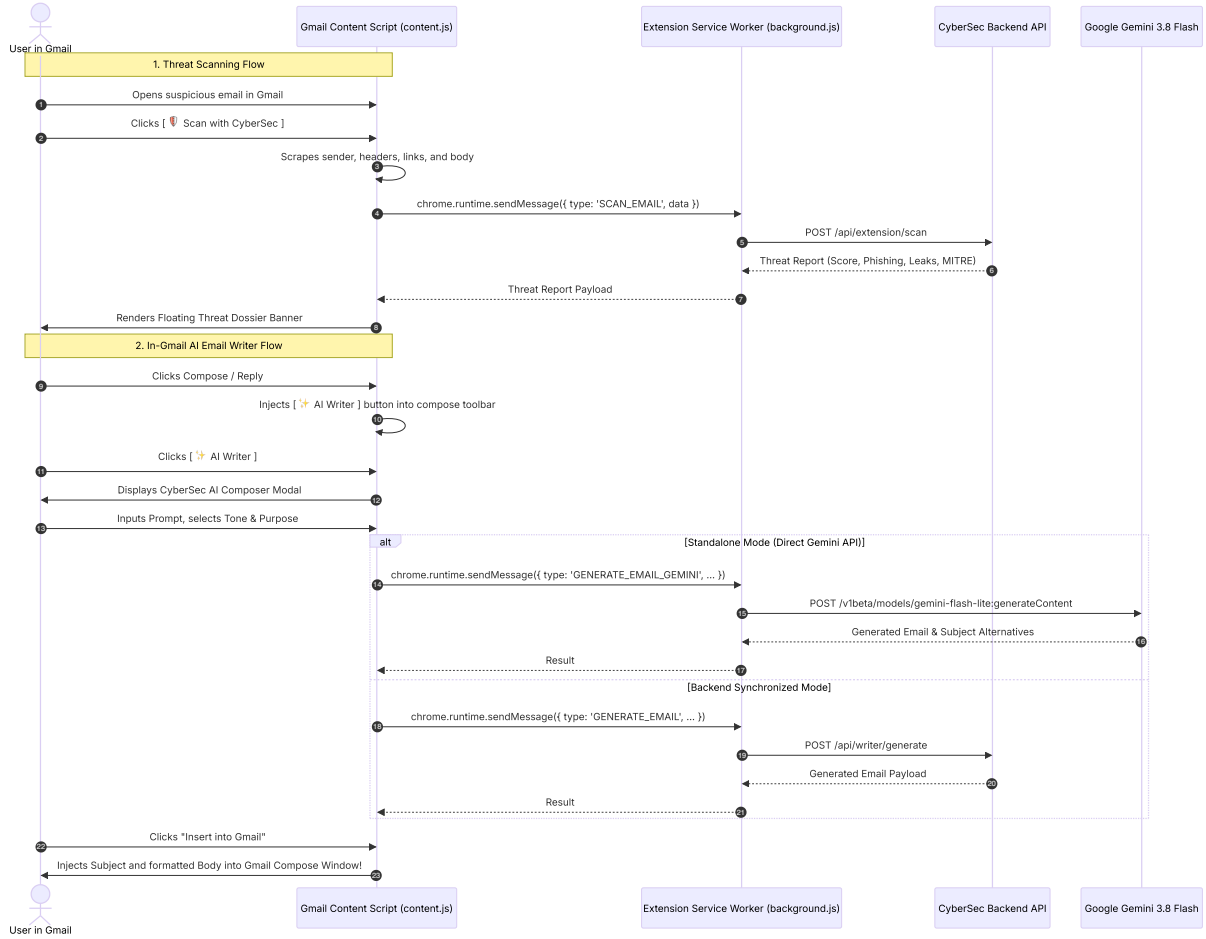
7. Chrome Extension: Manifest V3 Architecture

The Chrome Extension (`extension/`) operates in two integrated modes:

1. **Threat & Phishing Radar**: Injects `[ Scan with CyberSec ]` directly into the Gmail email header bar. Clicking it extracts the active email body, sender, headers, and links, sends them to the local/cloud API, and renders an interactive security dossier right above the message.

2. **In-Gmail AI Email Writer**: Injects an `[ ✨ AI Writer ]` action button into Gmail's compose window (both popups and inline reply threads). Clicking it opens an interactive modal right inside Gmail, allowing users to draft authentic, professional emails using Google Gemini or the local backend.

ARCHITECTURE DIAGRAM **Figure 4: Chrome Extension Manifest V3 Threat Radar & In-Gmail AI Composer Sequence**

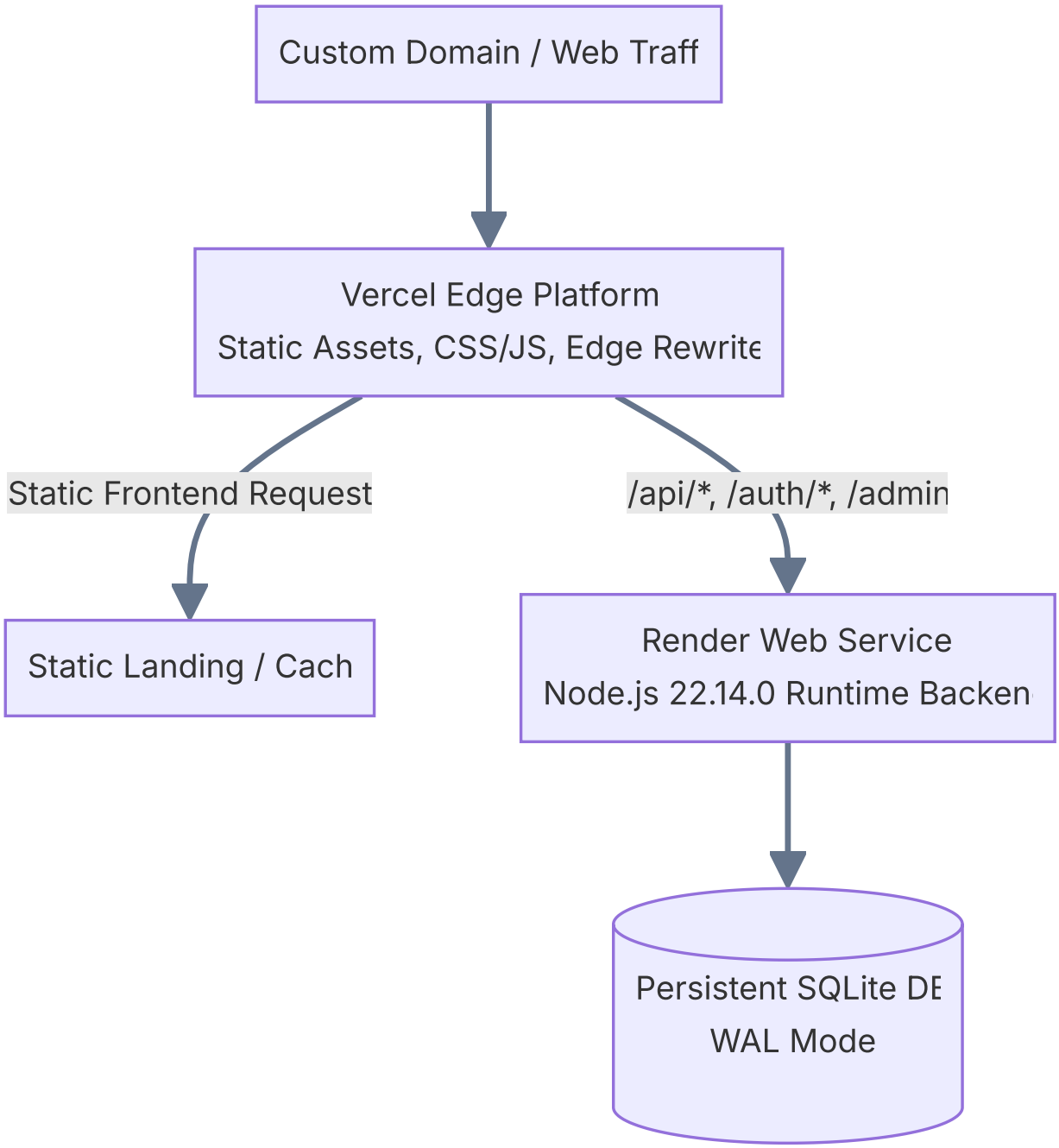


8. Deployment, Infrastructure & Production Readiness

8.1 Dual Deployment Architecture

The project is architected for frictionless dual deployment:

ARCHITECTURE DIAGRAM **Figure 5: Edge Frontend (Vercel) & Daemon Backend (Render) Dual Deployment Topology**



1. **Backend Daemon (Render)**:
   - Configured via `render.yaml`.
   - Runs full-stack Node.js server with active background scan daemon.
   - Handles OAuth handshakes, IMAP connections, and persistent SQLite database.

2. **Frontend Edge (Vercel)**:
   - Configured via `vercel.json`.
   - Serves high-speed cached landing page assets (`public/css/`, `public/js/`).
   - Automatically proxies dynamic routes (`/api/*`, `/auth/*`, `/admin/*`) to the Render backend.

8.2 Environment Variables Inventory

Variable Name	Required?	Default / Example	Purpose
`PORT`	Optional	`3000`	Port for Express HTTP listener.
`NODE_ENV`	Recommended	`production`	Enables production cookie flags and suppresses verbose logging.
`SESSION_SECRET`	**Strictly Required**	`(Random 64-char Hex)`	Cryptographic key for session signing and AES token encryption.
`ADMIN_INVITE_CODE`	Optional	`CYBERSEC-ADMIN-2026`	Secret pass-code granting `admin` role during user registration.
`GOOGLE_CLIENT_ID`	Optional	`*.apps.googleusercontent.com`	Google Cloud OAuth2 Client ID for Gmail API scanner.
`GOOGLE_CLIENT_SECRET`	Optional	`GOCSPX-*`	Google Cloud OAuth2 Client Secret.
`GOOGLE_REDIRECT_URI`	Optional	`http://localhost:3000/auth/google/callback`	OAuth2 callback redirect URL.
`GEMINI_API_KEY`	Optional	`AIzaSy...`	API key enabling Gemini 2.0 / 3.8 Flash email generation.
`OPENAI_API_KEY`	Optional	`sk-...`	Optional API key enabling GPT-4o email generation.
`FRONTEND_URL`	Optional	`https://your-domain.vercel.app`	Configures CORS access for decoupled frontends.

9. Verification & Local Execution Guide

9.1 Prerequisites

- **Node.js**: Version `⩾ 22.5.0` (Verify with `node -v`).
- **NPM**: Version `⩾ 10.0.0`.
- **Browser**: Google Chrome, Brave, Microsoft Edge, or Opera (for Chrome Extension testing).

9.2 Step-by-Step Installation

```bash
# 1. Clone repository and navigate to root directory
cd /Users/arpi85222707gmail.com/Documents/cybersec

# 2. Install dependencies
npm install

# 3. Initialize & Seed Database (Creates admin: admin / changeme123)
npm run seed

# 4. Start local development server with file watching
npm run dev
```

Visit the application at:

- **Landing Page**: `http://localhost:3000`
- **Admin Login**: `http://localhost:3000/login` (*Credentials:* `admin` / `changeme123`)
- **Email Threat Radar**: `http://localhost:3000/admin/email-scanner`
- **Email Classifier Studio**: `http://localhost:3000/admin/email-classifier`
- **AI Email Writer Studio**: `http://localhost:3000/admin/email-writer`

9.3 Installing the Chrome Extension

1. Open Google Chrome and navigate to `chrome://extensions`.
2. Toggle on **Developer mode** in the top-right corner.
3. Click **Load unpacked** in the top-left corner.
4. Select the `extension/` directory from this repository.
5. Open Gmail to test both threat scanning and the AI email writer!

---

10. Conclusion & Strategic Value

The **CyberSec Platform** provides an end-to-end cybersecurity management and threat response system. By pairing client-facing CRM lead acquisition with deep SOC telemetry, automated MITRE ATT&CK incident mapping, multi-class threat classification, and in-Gmail AI productivity tools, the project delivers enterprise-grade utility with minimal infrastructure footprint and zero native dependency friction.